

Connectivity

How do IoT devices talk?

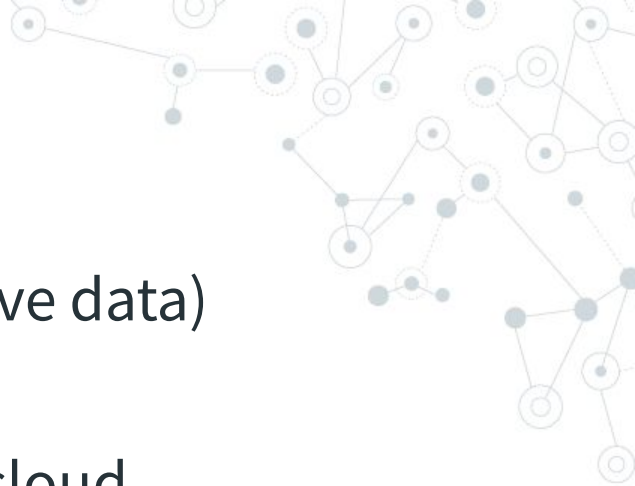
Need for connectivity

Why?

Where?

What are the possible options?

How to choose the correct one?



Need for connectivity

Why? To communicate (transfer and receive data)

Where? IoT device, controller, edge/hub, cloud

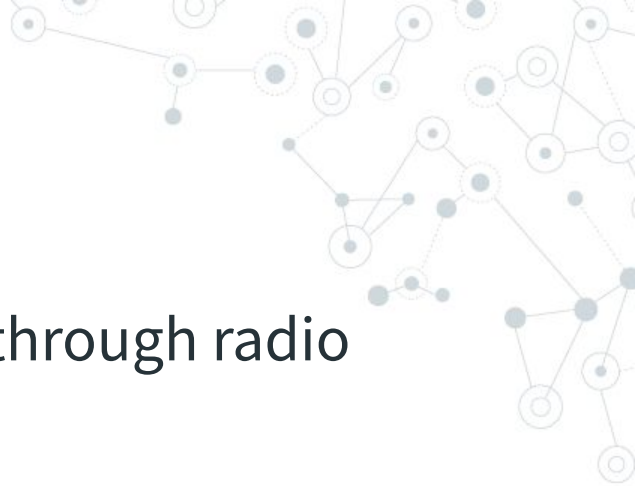
What are the possible options? WiFi, ZigBee, BLE



How to choose the correct one? Range, Bandwidth, cost, flexibility, performance, power

What do we need for connectivity?

- Wireless Transmission: A way to send data (bits) wirelessly (Radio processor).
- Data management: A method to send arbitrary data through the above mechanism (Techniques to manage the processor).



Wireless Transmission

Special hardware that can communicate through radio waves.

- WiFi
- BLE (Bluetooth)
- ZigBee

We need frequency and coordination.



WIFI: Frequency



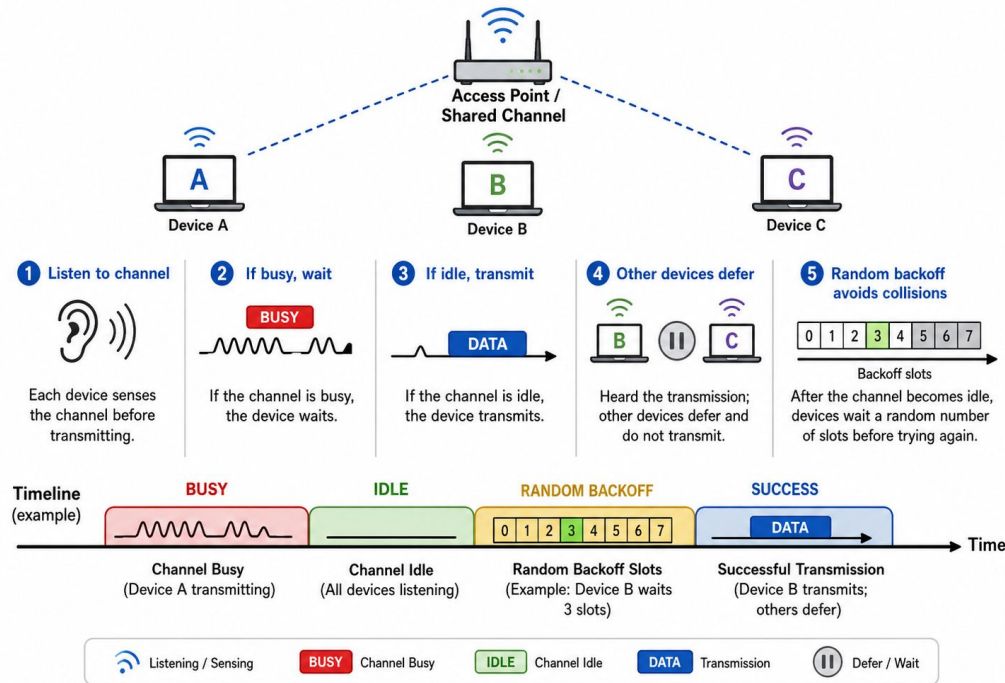
Wi-Fi band	Approx. frequency range	Typical standards	Main characteristics
 2.4 GHz	2.400–2.4835 GHz	802.11b/g/n, Wi-Fi 4/6	Longest range, better wall penetration, more interference
 5 GHz	roughly 5.15–5.85 GHz depending on country	802.11a/n/ac/ax, Wi-Fi 5/6	Higher capacity and speed, shorter range than 2.4 GHz
 6 GHz	5.925–7.125 GHz in the U.S.	Wi-Fi 6E, Wi-Fi 7	Very large spectrum, low congestion, shorter range
 60 GHz	roughly 57–71 GHz in the U.S.	802.11ad/ay, WiGig	Extremely high bandwidth, very short range, poor wall penetration
 Sub-1 GHz	around 900 MHz, region dependent	802.11ah / Wi-Fi HaLow	Long range, low power, designed for IoT



Rule of thumb

- **Lower frequency** → longer range, better penetration, generally lower available bandwidth
- **Higher frequency** → shorter range, poorer penetration, generally more available bandwidth

WIFI: Coordination : CSMA (Listen Before Talk)



WIFI: Data Management

We know how to send bits.

Now we need to send arbitrary data through WIFI --- How?

- Applications have different requirements.
- Avoid duplicate work.

WIFI: Data Management

Sending data has multiple steps.

Example:

- Finding and contacting the destination.
- Setting up the connection.
- Sending the data.
- Acknowledging the data received.
- Ending the connection.

WIFI: Data Management

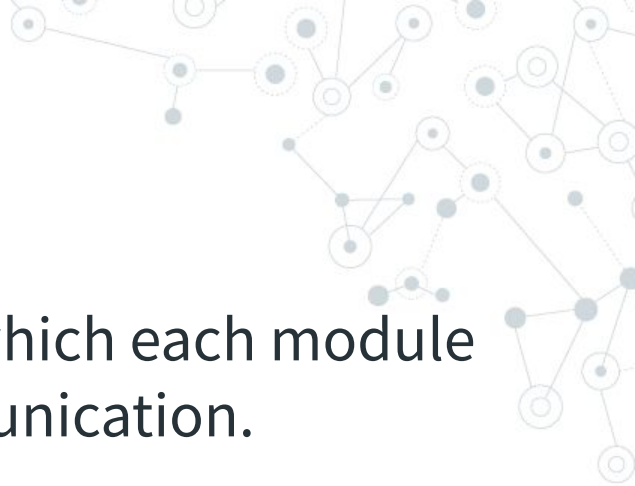
Sending data has multiple steps.

Example:

- Finding and contacting the destination --- Depends on network organization.
 - Setting up the connection --- Depends on the target system.
 - Sending the data --- Depends on application.
 - Acknowledging the data received --- Depends on the target system.
- Ending the connection --- Depends on network organization.



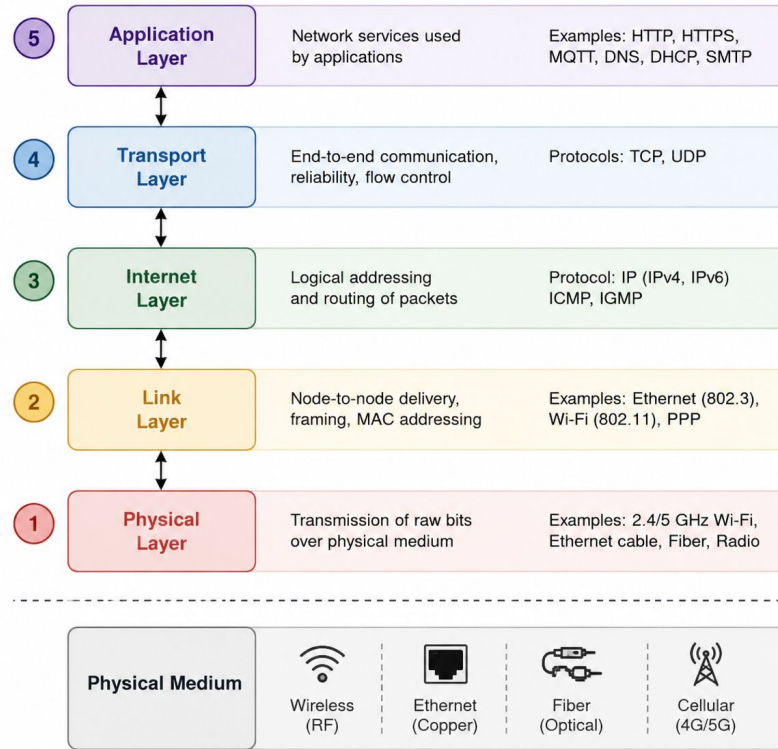
Separation of Concerns: A complex system should be divided into components that can be handled, managed and controlled independently.



Data management: Network Stack

A modular (i.e., layered) organization in which each module takes care of specific aspect of the communication.

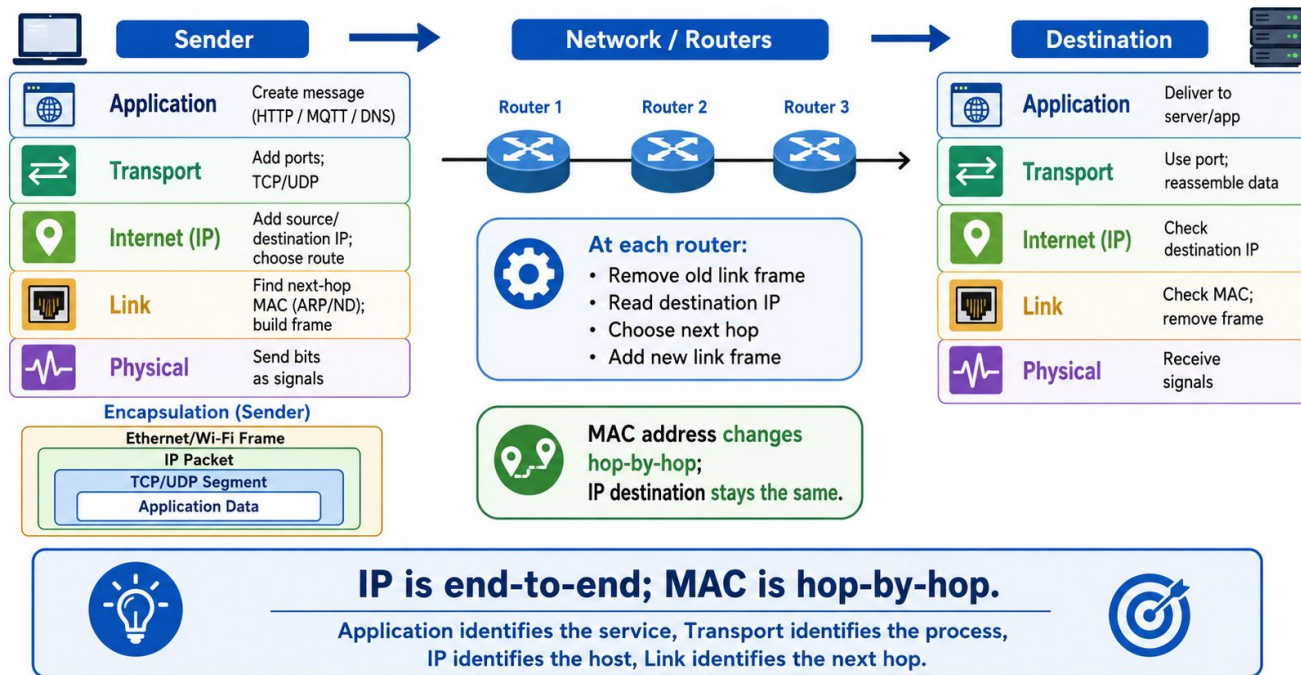
IP Network Stack (TCP/IP Model)



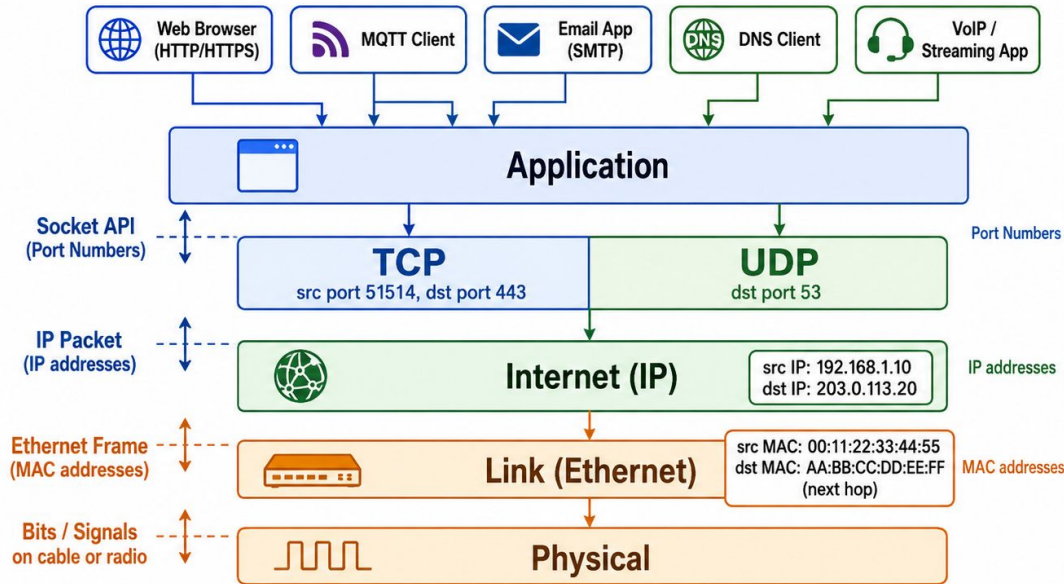
IP Network Stack

- Sending Data: Each layer receives data from upper layer, adds the upper layer specific information and sends to the lower layer.
- Receiving Data: A layer receives data from lower layer, reads information about which upper layer to transfer the data and sends the data to the corresponding upper layer component.

How Data Travels Through the IP Stack



How Applications Use the IP Stack



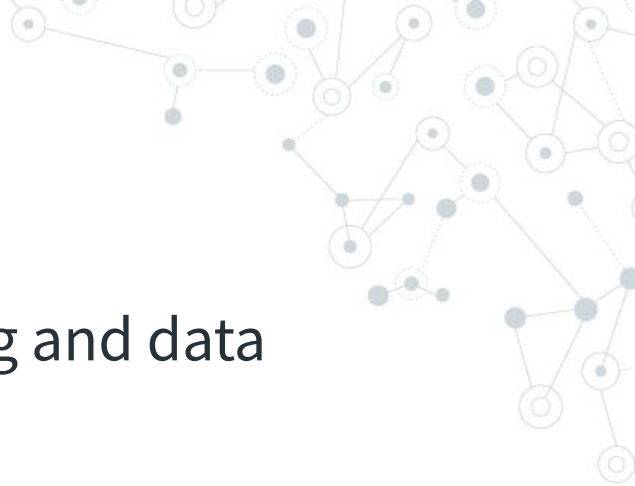
Who talks to whom?

- 1 Applications talk to TCP or UDP using port numbers.
- 2 TCP and UDP hand segments/datagrams to IP.
- 3 IP routes packets using IP addresses.
- 4 IP sends packets to Ethernet for local delivery using MAC addresses.

 **Ports** identify the application process. **IP addresses** identify the end host. **MAC addresses** identify the next hop on the local link.

IP Stack

- Can we have multiple IP addresses for a single ethernet card?
- Can we have multiple ethernet addresses for a single WIFI chip?
- Can we have multiple ports for a single IP address?



Bluetooth/ Bluetooth Low Energy (BLE)

Frequency: ~2.4 GHz (split into advertising and data channels)

BL v/s BLE:

- Continuous radio v/s Intermittent radio operation (BLE).
 - Small number of channels in BLE.
 - BLE Optimized for short bursts of data.
- 

BLE: Channel Coordination

How does BLE or Bluetooth work?

BLE: Channel Coordination

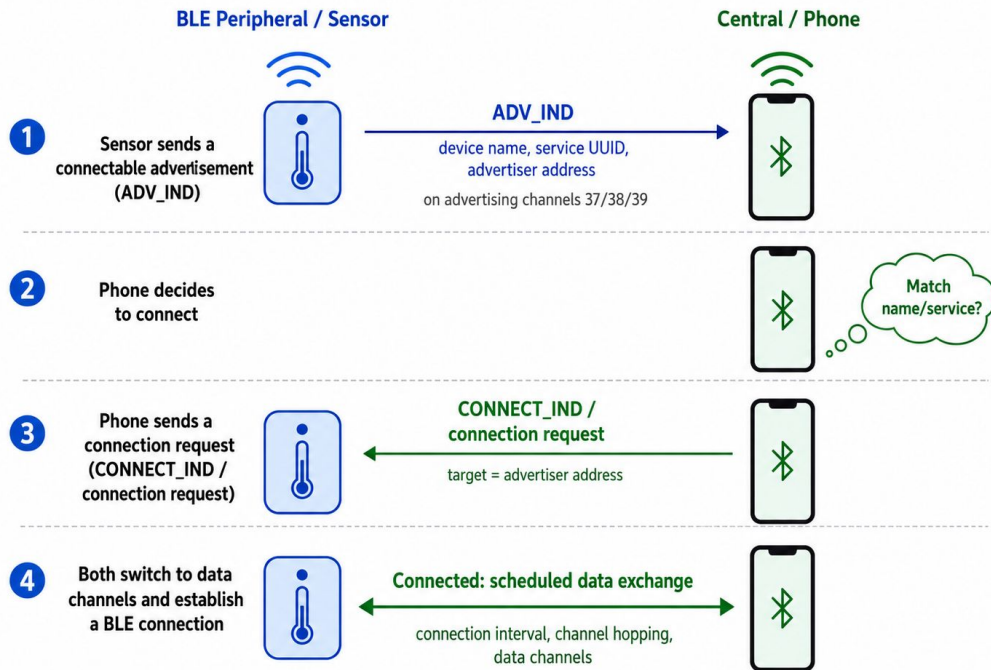
How does BLE or Bluetooth work?

- Two roles: Peripheral (device) and Central.
- Peripheral (e.g., Temperature sensor):
 - Wants to be connected and send data.
- Central (e.g., Phone):
 - Detects peripheral and initiates and manages the connection.

BLE: Channel Coordination

- Peripheral advertises (broadcasts) on known channels (i.e., frequencies).
- Central tries to connect to it.
- If connection is successful, they switch to data channels.

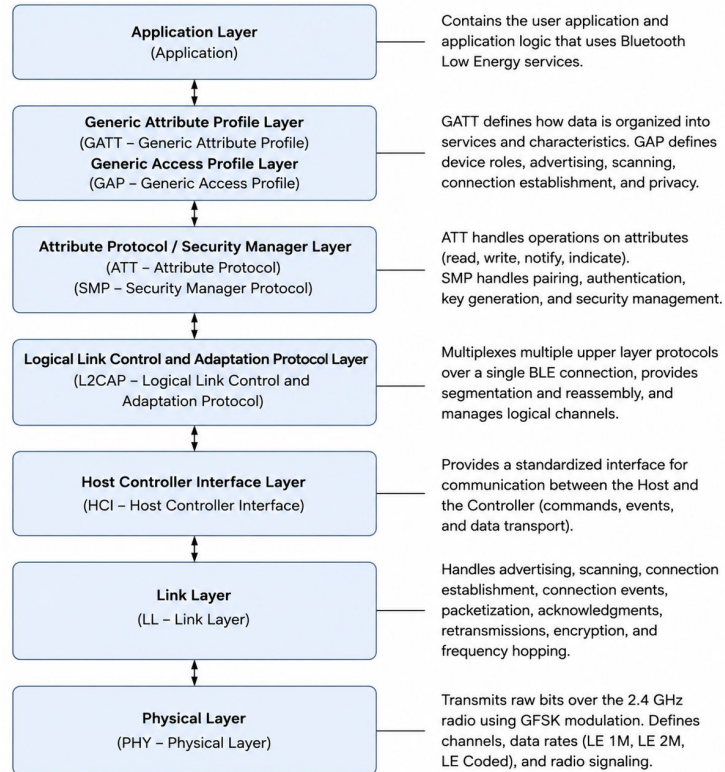
BLE: Advertising to Connection



BE/BLE Pairing

A mechanism to establish permanent connection using secrets --- more about this in next week.

BL/BLE Network Stack



BL/BLE Network Stack

- Why separate network stack? Can't we have IP stack over bluetooth?

ZigBee

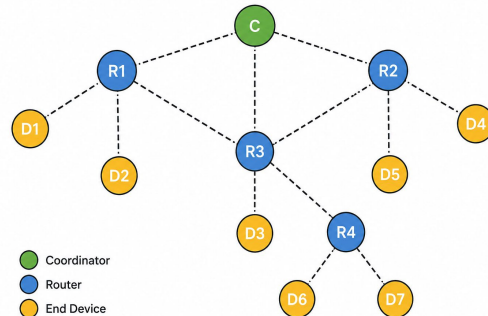
- ~2.4 GHz (~15 channels).
- Enables small personal area networks.

ZigBee: Coordination

Carrier Sense Multiple Access (CSMA) --- similar to WIFI.

ZigBee: Network Setup and Management

- **Co-ordinator:** Sets up network and assigns network addresses.
- **Router:** Forwards packet to other devices or routers.
- **Device:** End points, communicating entities.



ZigBee: Network Setup

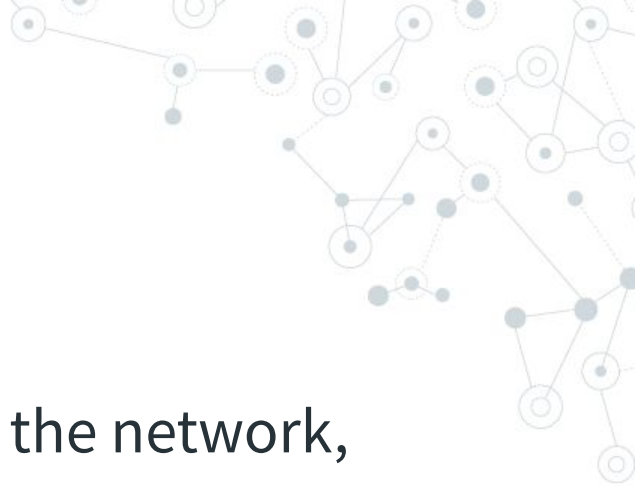
Co-ordinator:

- Scans for free channel and selects a free channel.
- Assigns a network ID (e.g., 0x4567)
- Looks (listens) for joining beacon.

ZigBee: Device Joining

Device:

- Send beacon request on each channel and wait for sometime.
- Coordinator sends response.
- Device joins the network --- an ID (address) is assigned to the device.



ZigBee: Router

Router:

- Same as coordinator but doesn't start the network, just joins an existing network.
- So, device may connect to router or directly to co-ordinator.

ZigBee: Communication

Usually communicated through coordinator (knows all the devices in the network).

Router and coordinator are awake all the time.

Devices usually sleep, wake up when:

- Need to send data.

- Received data.

ZigBee: Routing

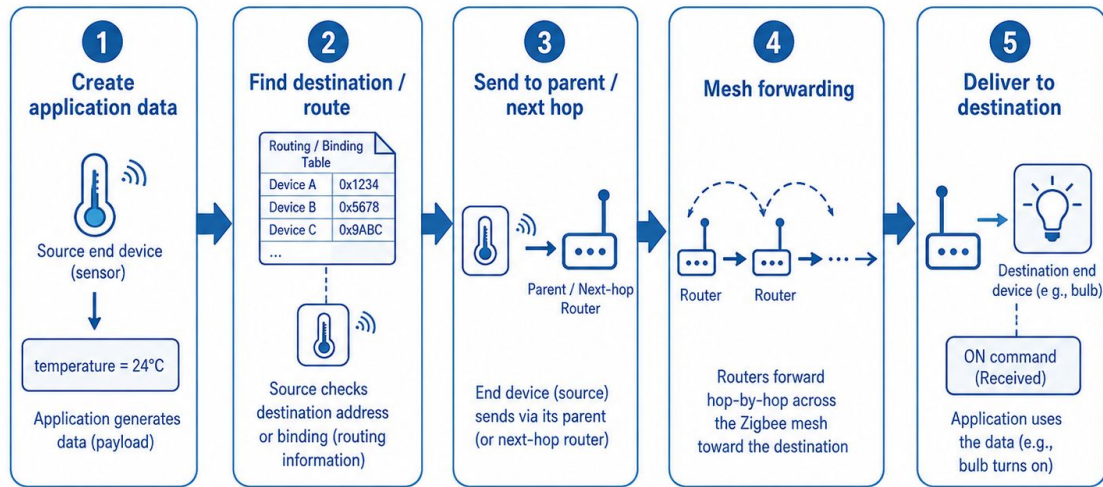
- Information maintained at router and coordinator.
- Similar to routing table:
 - Device ID to 64-bit IEEE address (directly connected devices).
 - Device ID to Router ID (other devices).

ZigBee: Sending Data

- There is no DNS in ZigBee, i.e., human readable name to network address.
- However, there is ZDO (ZigBee Device Object) discovery, where you can search for devices [based on capabilities](#).

You can find devices which has certain capability and provides a service.

ZigBee: Sending Data



Typical path: Source device → parent router → mesh routers → destination (or destination's parent router)

ZigBee: Receiving Data

Device sleeps!!



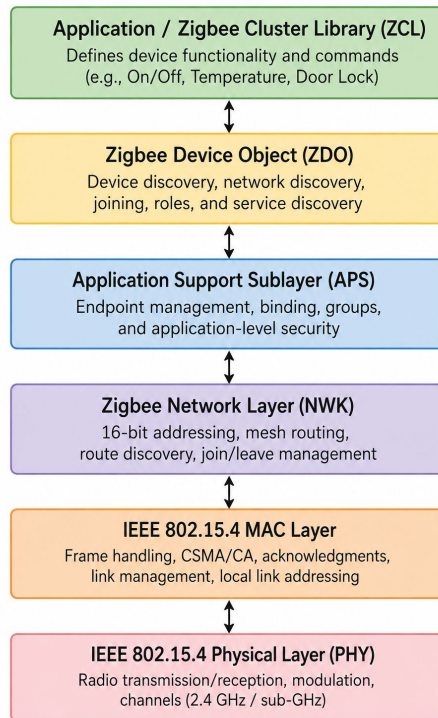
ZigBee: Receiving Data

Device sleeps!!

Messages buffered at router/coordinator:

- Device needs to pull messages from router/coordinator.

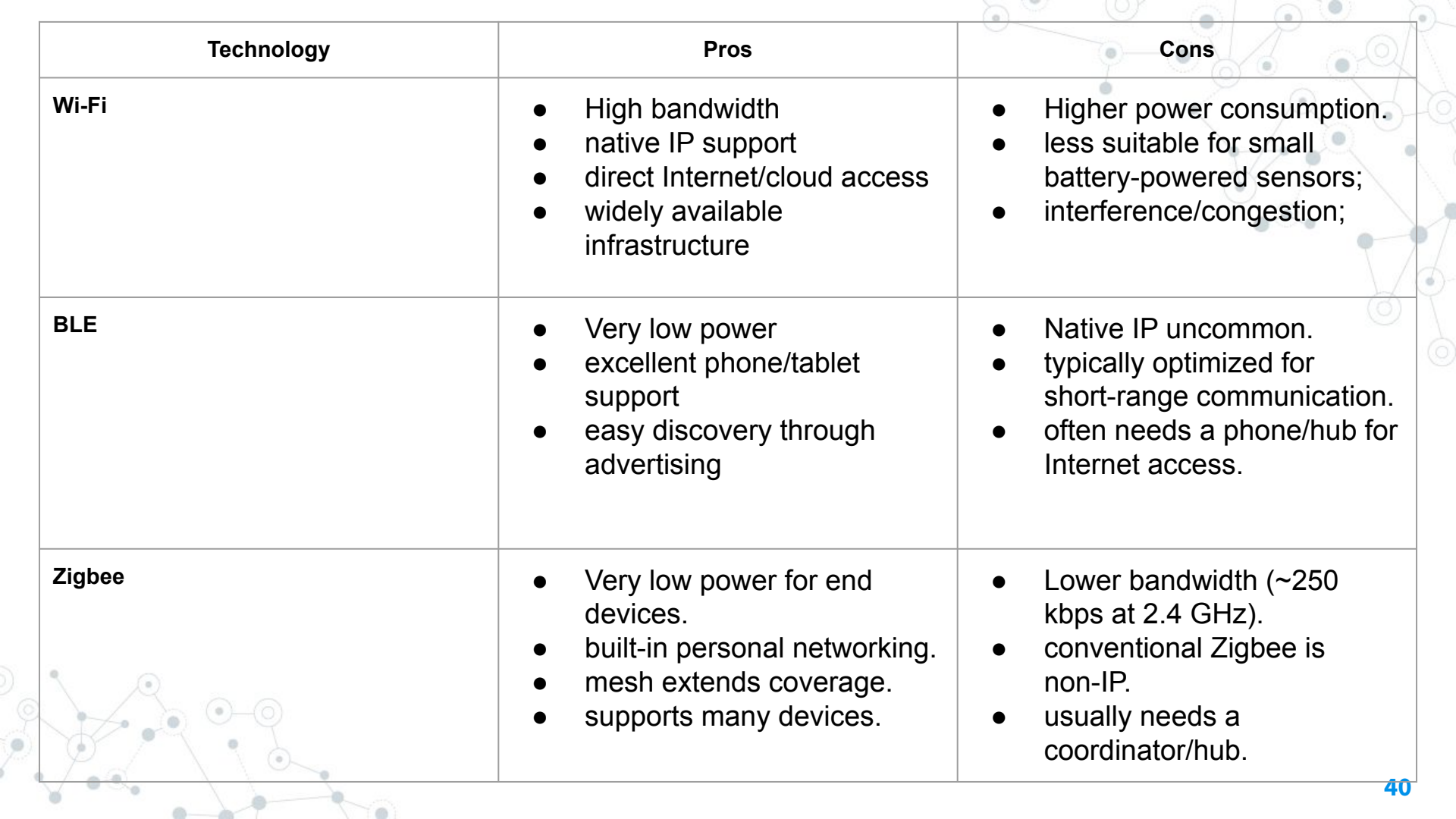
ZigBee Protocol Stack





Which connection type to choose?

Is there a best connection type?



Technology	Pros	Cons
Wi-Fi	• High bandwidth • native IP support • direct Internet/cloud access • widely available infrastructure	• Higher power consumption. • less suitable for small battery-powered sensors; • interference/congestion;
BLE	• Very low power • excellent phone/tablet support • easy discovery through advertising	• Native IP uncommon. • typically optimized for short-range communication. • often needs a phone/hub for Internet access.
Zigbee	• Very low power for end devices. • built-in personal networking. • mesh extends coverage. • supports many devices.	• Lower bandwidth (~250 kbps at 2.4 GHz). • conventional Zigbee is non-IP. • usually needs a coordinator/hub.

How to choose connection type?

- Range: local (e.g., within home)/remote (across the world).
- Power Constraints: Small Battery powered/ Rechargeable battery/Constant power.
- Infrastructure Support: Existing Hub/Edge devices or cloud.
- Desired Bandwidth: Few MB v/s Hundreds MBs
- Reliability: Require constant connection?
- Usability: Plug and play v/s medium effort.
- Security: Detection === Access?
- Cost: \$\$\$?

Choose Appropriate Connection Type

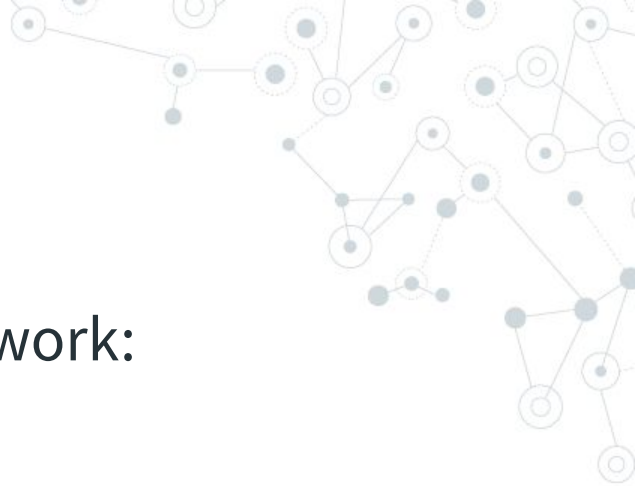
- IoT device 1: battery powered, no infrastructure support, local access is fine.
- IoT device 2: AC powered, has edge infrastructure, need remote access.
- IoT device 3: AC powered, need remote access, has cloud infrastructure.



What about applications?

How should applications send/receive data?

- Format of the data.
 - Interpreting the data.
 - Transmitting the data.
- 



Application Layer Protocols

Ways for applications to interact over network:
HTTP, MQTT, WebSocket, Matter.

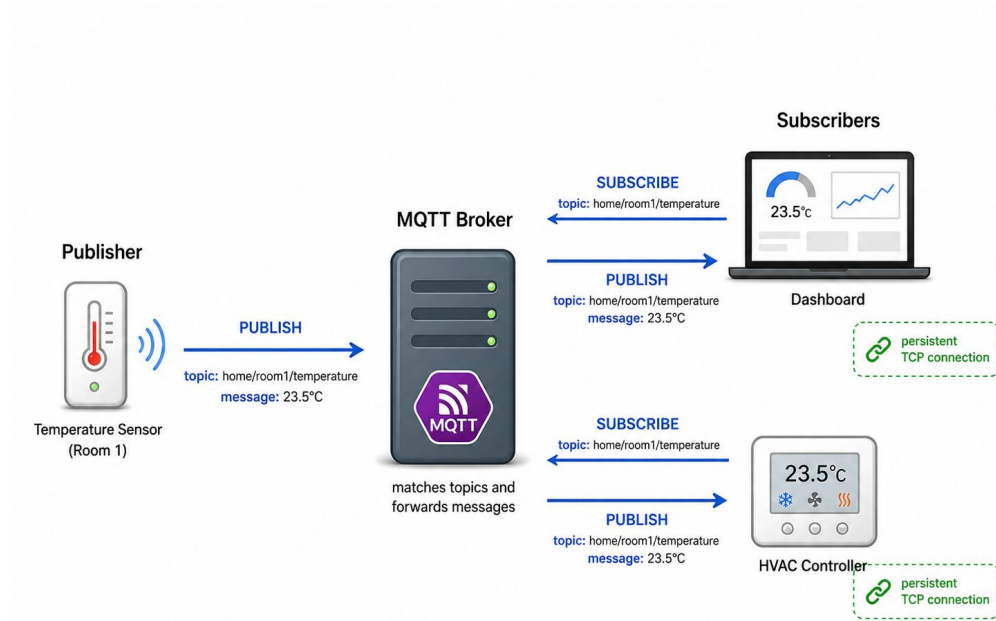
HTTP

- Standard ways to access resources: GET/PUT/POST/DELETE.
- Easy to use frameworks: RESTApis.
- CoAP (Constraint Application Protocol):
 - Similar to HTTP (but smaller packets, customized to IoT devices)

```
POST /deviceCommand
Content-Type:
application/json
```

```
{
  "action": "setTemperature",
  "deviceId": "thermostat-17",
  "value": 21
}
```

MQTT: Message Queuing Telemetry Transport



WebSocket

Allows bidirectional persistent connection, e.g., real-time chat through a ring camera.

Matter

Application protocol for handling
common home IoT activities.
e.g., pairing, etc.



The Foundation for Connected Things

One protocol to connect compatible devices and systems with one another. Smart home devices should be secure, reliable, and seamless to use. And with Matter, they are.

[Download Matter Specification](#)

Need to buy license if you want your device to be Matter compliant (Similar to ARM).

Can use for experimentation/projects/local hobby/etc.



Which application protocol to use?

It depends!!

